

1. Set Up the Environment

```
TA2_Vergara.py > ...
1  import random
2
3  # TASK 1: SET UP THE ENVIRONMENT
4  class TwoRoomEnvironment:
5      """Represents the 2-room environment (Rooms A and B)."""
6      def __init__(self, state_a="Clean", state_b="Clean"):
7          # Initializing two rooms using a Python dictionary
8          self.rooms = {
9              'A': state_a.capitalize(),
10             'B': state_b.capitalize()
11         }
12
13     def is_dirty(self, room: str) -> bool:
14         """Function to check if a specified room is dirty."""
15         return self.rooms.get(room) == "Dirty"
16
17     def clean_room(self, room: str):
18         """Function to clean a specified room."""
19         self.rooms[room] = "Clean"
```

2. Implement Rule-Based Agent

```
21  # TASK 2: IMPLEMENT THE RULE-BASED AGENT
22  class RuleBasedVacuumAgent:
23      """Represents the rule-based reflex agent."""
24      def __init__(self, initial_location='A'):
25          self.current_location = initial_location.upper()
26
27      def move(self):
28          """Switches the agent's location between Room A and Room B."""
29          self.current_location = 'B' if self.current_location == 'A' else 'A'
30
31      def perceive_and_act(self, env: TwoRoomEnvironment) -> str:
32          """Perceives environment state and acts according to predefined rules."""
33          curr = self.current_location
34
35          # Rule 1: If current room is dirty, clean it.
36          if env.is_dirty(curr):
37              env.clean_room(curr)
38              return f"Cleaned Room {curr}"
39
40          # Rule 2: If current room is clean, move to the other room.
41          else:
42              self.move()
43              return f"Moved to Room {self.current_location}"
44
```

3. Run the Simulation

```
# TASK 3: RUN THE SIMULATION (2 ROOMS) WITH RETRY OPTION
def run_task3_simulation():
    while True:
        print("\nTASKS 1, 2, & 3: 2-ROOM AGENT SIMULATION")

        # Task 1: User inputs for environment setup
        state_a = input("Enter initial state for Room A (Clean/Dirty): ").strip().capitalize()
        state_b = input("Enter initial state for Room B (Clean/Dirty): ").strip().capitalize()
        start_loc = input("Enter agent starting room (A/B): ").strip().upper()
        if start_loc not in ['A', 'B']:
            start_loc = 'A'

        env = TwoRoomEnvironment(state_a=state_a, state_b=state_b)
        agent = RuleBasedVacuumAgent(initial_location=start_loc)

        try:
            steps = int(input("Enter number of simulation steps to run: "))
        except ValueError:
            print("Invalid input! Defaulting to 4 steps.")
            steps = 4

        print("\n" + "-" * 65)
        print(f"INITIAL STATE: {env.rooms} | AGENT START: Room {agent.current_location}")
        print("-" * 65)

        # Task 3: Simulation loop
        for step in range(1, steps + 1):
            loc_before = agent.current_location
            action = agent.perceive_and_act(env)

            print(f"\nStep {step}:")
            print(f"  Agent Location Before Action : Room {loc_before}")
            print(f"  Action Taken                     : {action}")
            print(f"  Current Environment State       : {env.rooms}")

        print("Simulation Complete.")

        # Retry prompt
        retry = input("\nDo you want to retry this 2-Room simulation? (y/n): ").strip().lower()
        if retry != 'y':
            print("Returning to main menu...\n")
            break
```

CS0065 TECHNICAL ASSESSMENT 2

1. Run Tasks 1, 2 & 3 (2-Room Simulation)
2. Run Bonus Task (3-Room Text-Grid Simulation)
3. Exit

Select an option (1/2/3):

Select an option (1/2/3): 1

TASKS 1, 2, & 3: 2-ROOM AGENT SIMULATION

Enter initial state for Room A (Clean/Dirty): Dirty

Enter initial state for Room B (Clean/Dirty): Dirty

Enter agent starting room (A/B): A

Enter number of simulation steps to run: 5

INITIAL STATE: {'A': 'Dirty', 'B': 'Dirty'} | AGENT START: Room A

Step 1:

Agent Location Before Action : Room A

Action Taken : Cleaned Room A

Current Environment State : {'A': 'Clean', 'B': 'Dirty'}

Step 2:

Agent Location Before Action : Room A

Action Taken : Moved to Room B

Current Environment State : {'A': 'Clean', 'B': 'Dirty'}

Step 3:

Agent Location Before Action : Room B

Action Taken : Cleaned Room B

Current Environment State : {'A': 'Clean', 'B': 'Clean'}

Step 4:

Agent Location Before Action : Room B

Action Taken : Moved to Room A

Current Environment State : {'A': 'Clean', 'B': 'Clean'}

Step 5:

Agent Location Before Action : Room A

Action Taken : Moved to Room B

Current Environment State : {'A': 'Clean', 'B': 'Clean'}

Simulation Complete.

4. Bonus Task

```

88 # OPTIONAL (BONUS TASK): 3 ROOMS WITH TEXT-GRID VISUALIZATION & RETRY
89 class ThreeRoomEnvironment:
90     """Environment for Rooms A, B, and C."""
91     def __init__(self, a, b, c):
92         self.rooms = {'A': a.capitalize(), 'B': b.capitalize(), 'C': c.capitalize()}
93
94     def is_dirty(self, room):
95         return self.rooms.get(room) == "Dirty"
96
97     def clean(self, room):
98         self.rooms[room] = "Clean"
99
100     def get_dirty_rooms(self):
101         return [r for r, s in self.rooms.items() if s == "Dirty"]
102
103
104 class BonusAgent:
105     """Agent that navigates 3 rooms and targets dirty rooms randomly."""
106     def __init__(self, start='A'):
107         self.location = start.upper()
108
109     def perceive_and_act(self, env: ThreeRoomEnvironment):
110         curr = self.location
111         if env.is_dirty(curr):
112             env.clean(curr)
113             return f"Cleaned Room {curr}"
114
115         dirty_rooms = [r for r in env.get_dirty_rooms() if r != curr]
116         if dirty_rooms:
117             self.location = random.choice(dirty_rooms)
118             return f"Moved to Dirty Room {self.location}"
119         else:
120             other_rooms = [r for r in ['A', 'B', 'C'] if r != curr]
121             self.location = random.choice(other_rooms)
122             return f"Patrolled to Clean Room {self.location}"
123
124

```

```

125 def render_grid(env_rooms, agent_loc):
126     """Evenly spaced 1x3 grid box with aligned boundary borders."""
127     row_strings = []
128     for r in ['A', 'B', 'C']:
129         status = env_rooms[r]
130         agent_str = "[*AGENT*]" if agent_loc == r else ""
131         content = f"Room {r}: {status} {agent_str}".strip()
132         row_strings.append(f"| {content:<24} ")
133
134     border = "+-----" * 3 + "+"
135     print(border)
136     print("".join(row_strings) + "|")
137     print(border)
138
139
140 def run_bonus_simulation():
141     while True:
142         print("\nBONUS TASK: 3-ROOM GRID SIMULATION")
143
144         state_a = input("Enter state for Room A (Clean/Dirty): ").strip().capitalize()
145         state_b = input("Enter state for Room B (Clean/Dirty): ").strip().capitalize()
146         state_c = input("Enter state for Room C (Clean/Dirty): ").strip().capitalize()
147         start_loc = input("Enter starting room (A/B/C): ").strip().upper()
148         if start_loc not in ['A', 'B', 'C']:
149             start_loc = 'A'
150
151         try:
152             steps = int(input("Enter number of simulation steps: "))
153         except ValueError:
154             print("Invalid input! Defaulting to 4 steps.")
155             steps = 4
156
157         b_env = ThreeRoomEnvironment(state_a, state_b, state_c)
158         b_agent = BonusAgent(start_loc)
159
160         print("\nInitial Grid:")
161         render_grid(b_env.rooms, b_agent.location)
162

```

```

163         for s in range(1, steps + 1):
164             act = b_agent.perceive_and_act(b_env)
165             print(f"\nStep {s}: {act}")
166             render_grid(b_env.rooms, b_agent.location)
167
168         print("\n" + "=" * 65)
169         print("Bonus Simulation Complete.")
170         print("=" * 65)
171
172         # Retry prompt
173         retry = input("\nDo you want to retry this 3-Room simulation? (y/n): ").strip().lower()
174         if retry != 'y':
175             print("Returning to main menu...\n")
176             break
177
178     # MAIN EXECUTION MENU
179     if __name__ == "__main__":
180         while True:
181             print("\nCS0065 TECHNICAL ASSESSMENT 2")
182             print("1. Run Tasks 1, 2 & 3 (2-Room Simulation)")
183             print("2. Run Bonus Task (3-Room Text-Grid Simulation)")
184             print("3. Exit")
185
186             choice = input("\nSelect an option (1/2/3): ").strip()
187
188             if choice == '1':
189                 run_task3_simulation()
190             elif choice == '2':
191                 run_bonus_simulation()
192             elif choice == '3':
193                 print("Exiting program. Goodbye!")
194                 break
195             else:
196                 print("Invalid selection. Please choose 1, 2, or 3.")

```

```

Do you want to retry this 2-Room simulation? (y/n): n
Returning to main menu...

```

```

CS0065 TECHNICAL ASSESSMENT 2
1. Run Tasks 1, 2 & 3 (2-Room Simulation)
2. Run Bonus Task (3-Room Text-Grid Simulation)
3. Exit

```

```

Select an option (1/2/3): █

```

Select an option (1/2/3): 2

BONUS TASK: 3-ROOM GRID SIMULATION

Enter state for Room A (Clean/Dirty): Dirty

Enter state for Room B (Clean/Dirty): Dirty

Enter state for Room C (Clean/Dirty): Dirty

Enter starting room (A/B/C): C

Enter number of simulation steps: 6

Initial Grid:

Room A: Dirty	Room B: Dirty	Room C: Dirty [*AGENT*]	
---------------	---------------	-------------------------	--

Step 1: Cleaned Room C

Room A: Dirty	Room B: Dirty	Room C: Clean [*AGENT*]	
---------------	---------------	-------------------------	--

Step 2: Moved to Dirty Room B

Room A: Dirty	Room B: Dirty [*AGENT*]	Room C: Clean	
---------------	-------------------------	---------------	--

Step 3: Cleaned Room B

Room A: Dirty	Room B: Clean [*AGENT*]	Room C: Clean	
---------------	-------------------------	---------------	--

Step 4: Moved to Dirty Room A

Room A: Dirty [*AGENT*]	Room B: Clean	Room C: Clean	
-------------------------	---------------	---------------	--

Step 5: Cleaned Room A

Room A: Clean [*AGENT*]	Room B: Clean	Room C: Clean	
-------------------------	---------------	---------------	--

Step 6: Patrolled to Clean Room C

Room A: Clean	Room B: Clean	Room C: Clean [*AGENT*]	
---------------	---------------	-------------------------	--

=====
Bonus Simulation Complete.
=====

Do you want to retry this 3-Room simulation? (y/n): █

Do you want to retry this 3-Room simulation? (y/n): n
Returning to main menu...

CS0065 TECHNICAL ASSESSMENT 2

1. Run Tasks 1, 2 & 3 (2-Room Simulation)
2. Run Bonus Task (3-Room Text-Grid Simulation)
3. Exit

Select an option (1/2/3): 3

Exiting program. Goodbye!